Daniil Sobol

P2P / Distributed Systems Engineer — Node.js, Protocols, High-Performance Backend Systems

mynameisdaniil.dev | sobol.daniil@gmail.com | t.me/mynameisdaniil | github.com/mynameisdaniil

Remote Worldwide | Current Location: Yerevan, Armenia | Open to relocation

Professional Summary

Distributed systems engineer with 20+ years building high-load backend platforms, P2P protocol components, and latency-sensitive infrastructure across blockchain, finance, and messaging systems. Hands-on experience with Node.js, Erlang/Elixir, Rust, C, asynchronous systems, unreliable networks, protocol-level engineering, and production services handling up to millions of messages per second. I bring systems-level judgment, performance focus, and operational ownership to teams building core networking, storage, and peer-to-peer infrastructure.

Core Expertise

- **Peer-to-peer and distributed systems:** Ethereum Portal Network, gossip protocols, decentralized data propagation, resource-constrained clients, unreliable networks, distributed state, replication, and fault tolerance.
- **High-performance backend engineering:** Node.js, Elixir / Erlang, Rust, Python, Go, C, event-driven architecture, asynchronous processing, real-time pipelines, APIs, microservices, and message-driven systems.
- **Networking and protocol engineering:** TCP/IP, UDP concepts, binary protocol design, serialization, cryptographic primitives, latency-sensitive systems, throughput optimization, backpressure, and failure isolation.
- **Production reliability:** observability, metrics, tracing, logging, deployment automation, CI/CD, incident analysis, replayability, operational visibility, and architecture ownership.
- **Infrastructure and storage:** Kubernetes, Docker, Terraform, AWS, PostgreSQL, Riak, RabbitMQ, managed databases, S3-compatible storage, cloud and on-prem deployments.
- **Engineering leadership:** technical strategy, architecture reviews, hiring, mentoring, cross-functional delivery, and leading teams through complex distributed-system work.

Experience

Zoopt | *CTO* | 2025–Present

- Led engineering across backend systems, infrastructure, integrations, internal platforms, and automation for a 200-store retail network.
- Converted architecture, security, legal, and infrastructure audits into 20 engineering improvements delivered across four teams.
- Established access controls, observability, audit trails, human approval flows, and fallback mechanisms for reliable production automation.
- Recruited, mentored, and coordinated 10 engineers while strengthening ownership across five legacy systems.
- Directed production engineering decisions across cloud infrastructure, CI/CD, backend services, operational monitoring, and cross-system integrations.

Clickavia | *Backend Engineer* | 2024–2025

- Built and launched a production support automation system that handled 80% of requests and reduced average first-response time by 90%, from five minutes to 30 seconds.
- Designed uncertainty-aware escalation and fallback flows, routing 20% of cases to human operators and preventing silent customer-facing failures.
- Instrumented distributed workflows with tracing and metrics for latency, escalation rate, operational performance, and cost.
- Delivered Elixir and Phoenix backend features supporting tens of thousands of users and thousands of daily bookings.
- Improved production reliability through clearer service boundaries, observability, and failure-aware application design.

Status | *Fluffy, Ethereum Portal Network Client* | 2023–2024

- Engineered protocol-level components for Fluffy, a resource-constrained Ethereum Portal Network client operating across unreliable peer-to-peer networks.
- Contributed to gossip protocol and History Network functionality supporting decentralized data propagation, retrieval, and availability.
- Worked on asynchronous processing, distributed state, resource efficiency, and fault-tolerant behavior in decentralized network conditions.
- Applied systems-level reasoning to P2P protocol implementation, network behavior, data replication, and constrained client performance.
- Contributed to open-source blockchain infrastructure in a production-grade distributed systems environment.

Revea | *Backend and AI Product Engineer* | 2022–2023

- Built production backend services connecting Python-based components with Elixir and Phoenix applications used by several thousand customers.
- Architected service boundaries for five complex workflows, improving failure isolation, debuggability, deployment safety, and operational visibility.
- Automated infrastructure with Terraform, reducing environment setup and deployment time from one day to 30 minutes.
- Led hardware prototyping and backend integration, connecting physical data collection with production application workflows.
- Delivered reliable interfaces between application services, data pipelines, infrastructure, and customer-facing systems.

Netronix Group | *Backend Engineer* | 2021–2022

- Improved a core production backend service, contributing to an 80% reduction in operational incidents.

- Delivered cloud-native backend services using Python, Elixir, Kubernetes, AWS, and Terraform.
- Strengthened maintainability, deployment practices, and operational reliability across application services and infrastructure.
- Worked across application code, infrastructure, observability, and production support for high-availability backend systems.

Curogram | *Engineering and Scaling Support* | 2019–2021
- Helped scale a healthcare communications platform 40-fold, from 50 to 2,000 clinics, during the COVID demand surge.
- Removed backend and operational bottlenecks, tripling system throughput while maintaining production reliability.
- Supported hiring and engineering coordination as the team expanded from four to 22 people.
- Improved scalability, incident response, and operational resilience for a fast-growing production communications platform.

United Traders | *Team Lead and Exchange Systems Engineer* | 2017–2019
- Led development of Erlang-based exchange services benchmarked at 100,000 messages per second per symbol and designed for a 1 ms response-latency target.
- Built and operated latency-sensitive, event-driven trading infrastructure with strict correctness, reliability, and throughput requirements.
- Improved fault isolation, replayability, and operational visibility across eight event-driven trading services.
- Mentored five engineers and directed architectural decisions for systems with a 99.7% availability requirement.
- Contributed to Rust-based exchange components requiring predictable performance, strict correctness, and low-level systems awareness.

Lodoss Team | *Lead Engineer and Project Lead* | 2012–2017
- Reduced platform complexity by consolidating 20 microservices into seven while growing the engineering team from two to seven.
- Built deployment and orchestration tooling that cut release time by more than 90%, from four to five hours to 15 minutes or less.
- Engineered messaging infrastructure handling up to 500,000 messages per second per service.
- Replaced MongoDB and Neo4j bottlenecks with a six-node Pregel-inspired distributed graph-processing system.
- Improved graph execution time 15-fold, from 1.5 seconds to 100 ms, across 1.3 million vertices and several million edges.
- Designed distributed data-processing and message-driven systems focused on throughput, fault isolation, and operational simplicity.

Lodoss Team | *Software Engineer* | 2010–2011
- Built production services using Node.js, RabbitMQ, and Riak for a distributed application platform.
- Helped evolve the platform from MVP to production adoption by dozens of partner development studios.
- Worked with modular backend services, asynchronous messaging, distributed storage, and production service integration.

Freelance | *Web Developer* | 2006–2010
- Delivered five end-to-end web and business applications, owning requirements, implementation, deployment, and launch.
- Built and maintained production applications across backend logic, database design, deployment, and customer-facing workflows.

Education

Master's Degree in Radio Engineering
Southern Federal University, Taganrog, Russia

Bachelor's Degree in Radio Engineering
Southern Federal University, Taganrog, Russia

Technical Skills

- **Languages** – JavaScript, Node.js, Python, Elixir, Erlang, Rust, Go, C
- **Distributed Systems** – peer-to-peer systems, Ethereum Portal Network, gossip protocols, decentralized data propagation, distributed state, replication, fault tolerance, unreliable networks
- **Networking and Protocols** – TCP/IP, UDP concepts, binary protocols, serialization, cryptographic primitives, latency optimization, throughput optimization, backpressure, asynchronous I/O concepts
- **Backend Engineering** – APIs, microservices, event-driven architecture, real-time pipelines, high-load services, message-driven systems, service boundaries
- **Performance and Reliability** – profiling, observability, tracing, metrics, logging, incident analysis, replayability, operational visibility, deployment safety
- **Infrastructure** – Kubernetes, Docker, Terraform, AWS, cloud deployments, on-prem deployments, S3-compatible storage, managed databases
- **Databases and Messaging** – PostgreSQL, Ecto, Riak, RabbitMQ, custom message pipelines
- **Leadership** – engineering leadership, hiring, mentoring, architecture ownership, cross-functional delivery, technical strategy

Selected Public Talks

Regular conference speaker on distributed systems, actor architectures, high-load systems, blockchain infrastructure, and engineering culture.
- **EthBelgrade** Portal Network and libportalnet: Building Lightweight Ethereum Protocol Implementations [EN]
- **F[Scala]** Exchanges and the Theory of Relativity: Applying Concepts from Physics to High-Load Distributed Systems.
- **Saint P Ruby Community / Rubyconf** Designing Actor Architectures for Scalable Concurrent Systems [RU]
- **Moscow Elixir Meetup** Why Elixir Couldn't Have Been Based on the JVM: A Look at Garbage Collection in the BEAM [RU]
- **FPconf** Why You Probably Should Re-Invent the Wheel in Erlang: Lessons from Building Real Systems